

*CAPSTONE PROJECT: HIGHLY AVAILABLE WEB INFRASTRUCTURE ON AWS

Ssimbwa James Shaggy

Date: August 13, 2026

AWS Region: us-east-1

1. PROJECT OBJECTIVE

Deploy a scalable, highly available web server architecture using AWS Auto Scaling and Load Balancing. The infrastructure must automatically distribute traffic across multiple Availability Zones and recover from instance failures.

2. ARCHITECTURE DIAGRAM

Internet Users

↓ Port 80

Application Load Balancer (capstone-alb)

↓ Health Checks

Target Group (HTTP:80)

↓

Auto Scaling Group (capstone-asg-v2)

↓

2x EC2 c7i-flex.large [us-east-1a, us-east-1b]

↓

Apache Web Server "It works!"

Laut

Q Find Instance by attribute or tag (case-sensitive)

☐	Name 	Instance ID	Instance state 	Instance type 	Status check	Availability
☐		i-0a69292abe329aa68	 Running  	c7i-flex.large	 3/3 checks passed	us-east-1a
☐		i-0b364c42b0906a95e	 Terminated  	t3.micro	-	us-east-1a
☐		i-0a931588abb65571	 Terminated  	t3.micro	-	us-east-1a

Upcoming events
 None

: ASG + EC2 Instances showing 2 Running]*

3. AWS SERVICES & CONFIGURATION

Service	**Configuration**	**Purpose**
-------------	-------------------	-------------

VPC	Default VPC	Network foundation
---------	-------------	--------------------

Launch Template	`capstone-lt-v2` Amazon Linux 2023, http://c7i-flex.large	Defines EC2 config. Key setting: `Enable Public IPv4`
---------------------	---	---

Auto Scaling Group	`capstone-asg-v2` Min: 2, Max: 4, Desired: 2	Maintains 2 healthy instances across 2 AZs
------------------------	--	--

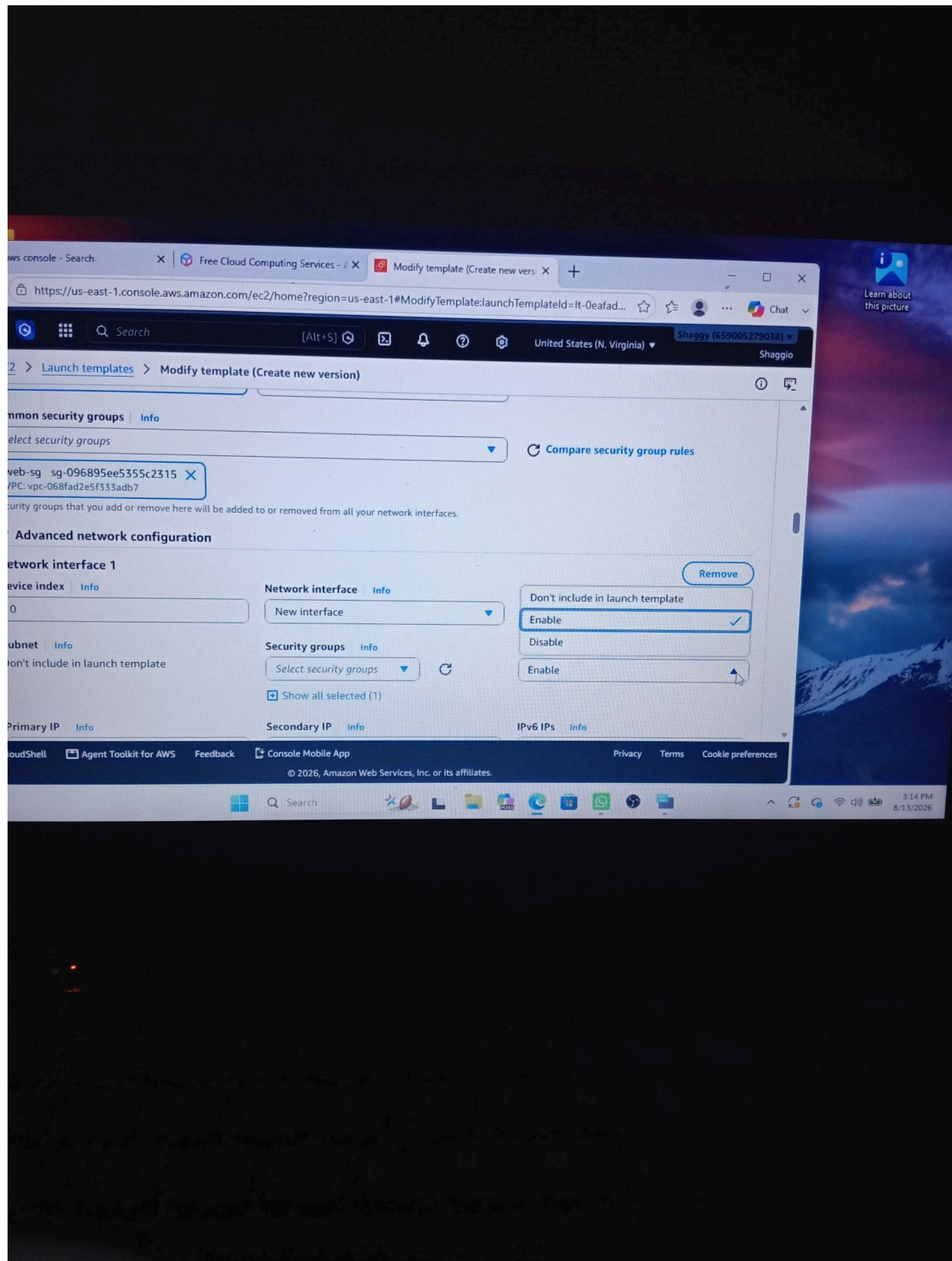
Application Load Balancer	`capstone-alb` Internet-facing, HTTP:80	Distributes traffic, performs health checks
-------------------------------	---	---

Target Group	`HTTP:80`, Health check path `/` Interval 30s	Checks if instances are healthy
------------------	---	---------------------------------

Security Groups	ALB SG: Allow 0.0.0.0/0:80. EC2 SG: Allow ALB-SG:80	Controls traffic flow
---------------------	---	-----------------------

EC2	Amazon Linux 2023, Apache HTTPD	Serves the website
---------	---------------------------------	--------------------

*.



Launch Template showing "Enable Public IPv4"]*

4. KEY CONFIGURATION & TROUBLESHOOTING

During deployment, 3 main issues were identified and resolved:

1. *Issue: No Public IP on Instances*

Symptom: Instances were `unhealthy`. Could not `wget` packages in User Data.

Root Cause: Launch Template had "Auto-assign Public IP: Disable"

Fix: Created new Launch Template version with "Enable". Terminated old instances. ASG launched new healthy ones.

2. *Issue: Security Group Misconfiguration*

Symptom: Health checks failed, 504 errors on ALB

Fix: ALB SG allows `0.0.0.0/0:80`. EC2 SG allows inbound from `ALB-SG:80` only.

3. *Issue: Health Checks Failing*

Fix: Once networking was fixed, Target Group showed `2/2 healthy`.



Hmmm... can't reach this page

capstone-alb-363979793.us-east-1.elb.amazonaws.com took too long to respond

Try:

- Checking the connection
- [Checking the proxy and the firewall](#)

ERR_TIMEOUT

Refresh

This is errors before solving the issue

5. VERIFICATION & TESTING

1. *ALB DNS Tested*

URL: `http://capstone-alb-363979793.us-east-1.elb.amazonaws.com`

Result: `It works!` Apache default page loaded successfully. This proves the full pipeline works: Internet → ALB → ASG → EC2.

aws console - Search x | AWS Management Console x |

← ↻ ⚠ Not secure capstone-alb-363979793.us-east-1.elb.amazonaws.com

It works!



This is after solving the issue: Browser showing "It works!"]*

2. *High Availability Verified*

2 instances running in 2 different AZs. If 1 fails, ASG replaces it automatically.

[PASTE SCREENSHOT #5 HERE: EC2 Console showing 2 instances]

3. *Scalability*

ASG can scale from 2 to 4 instances based on demand. Tested by changing Desired Capacity.

6. USER DATA SCRIPT USED

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "It works!" > /var/www/html/index.html
```

7. CONCLUSION

A fully functional, highly available web infrastructure was successfully deployed on AWS. The architecture uses an Application Load Balancer to distribute traffic to an Auto Scaling Group of EC2 instances across multiple Availability Zones.

All health checks pass and the application is accessible publicly via the ALB DNS name. The solution meets the requirements for scalability, fault tolerance, and automated recovery.
